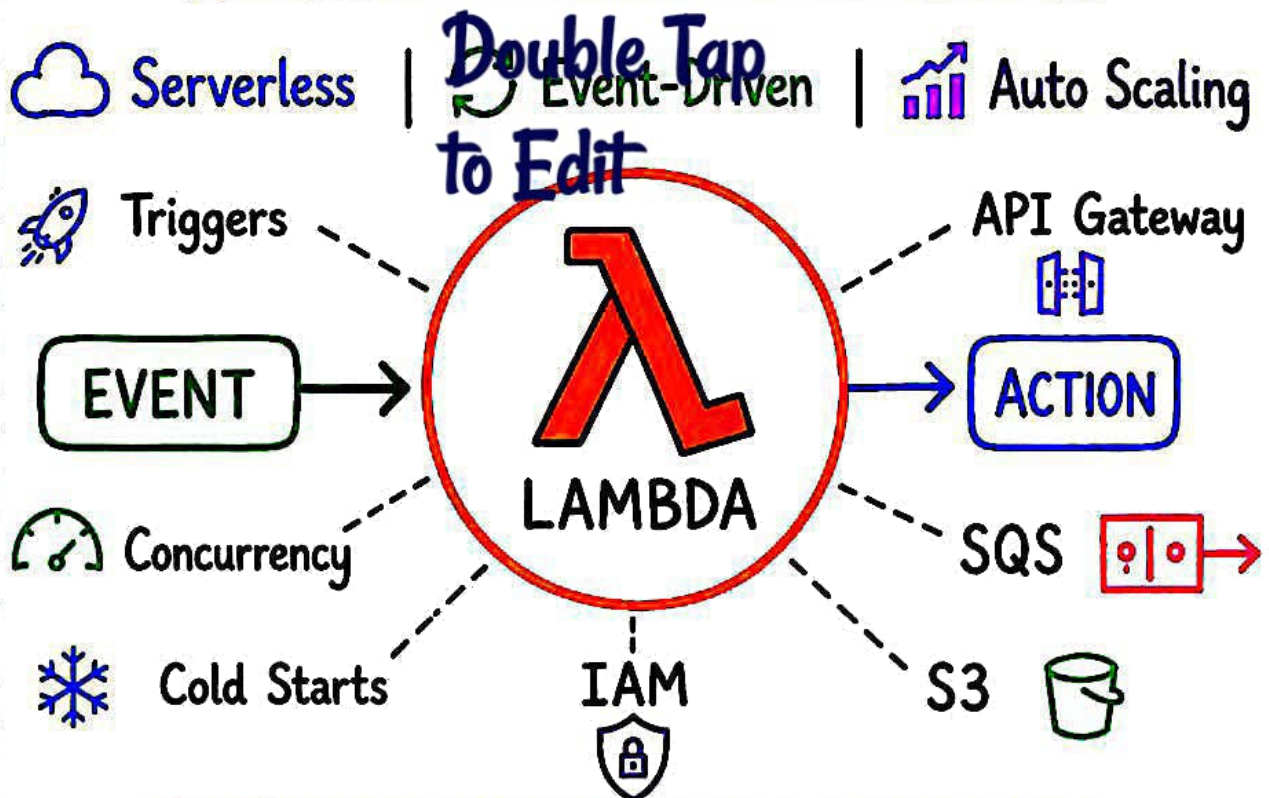




AWS LAMBDA COMPLETE NOTES

BEGINNER → ADVANCED

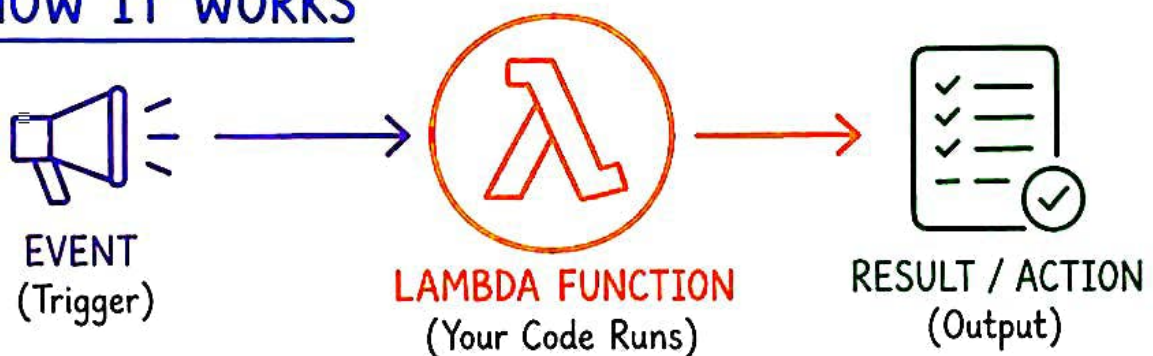


EVERYTHING YOU NEED TO KNOW
ABOUT AWS LAMBDA

② WHAT IS AWS LAMBDA?

AWS Lambda is a serverless compute service that runs your code in response to events without provisioning or managing servers.

HOW IT WORKS



KEY HIGHLIGHTS

- ✓ No server management
- ✓ Event-driven
- ✓ Automatic scaling
- ✓ High availability built-in
- ✓ Pay only for what you use
- ✓ Supports multiple runtimes (Python, Node.js, Java, Go, .NET, etc.)
- ✓ Integrates with many AWS services



YOU WRITE THE CODE.
AWS MANAGES THE INFRASTRUCTURE.

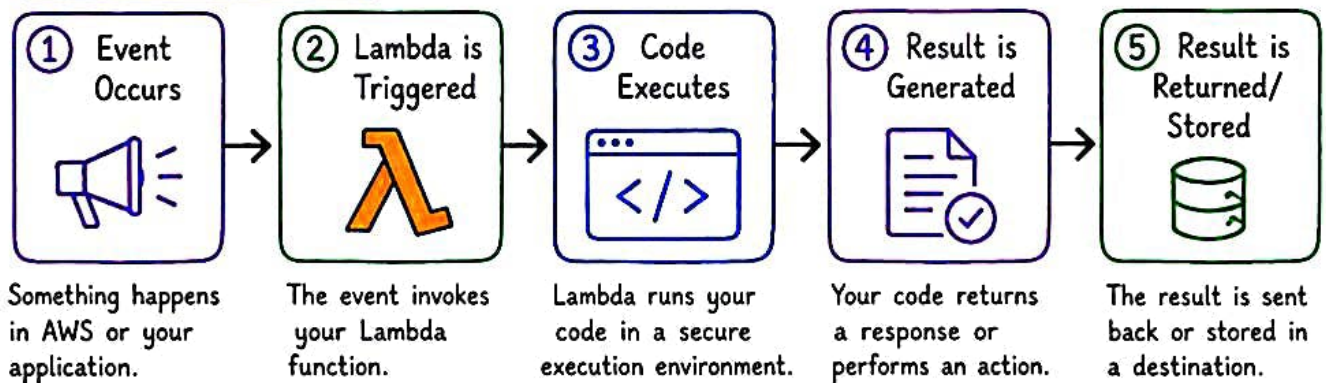


LAMBDA = SERVERLESS + EVENT-DRIVEN + AUTO SCALING

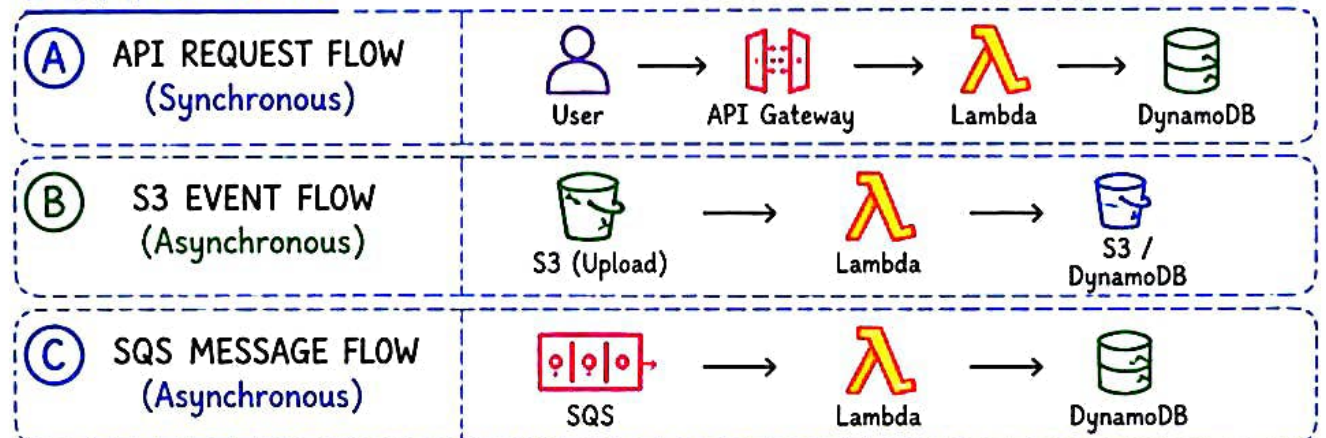
③ HOW LAMBDA WORKS

Lambda runs your code in response to events.
It is event-driven and automatically manages the infrastructure.

EXECUTION FLOW



EXAMPLE FLOWS



KEY POINT

You focus on writing code.
AWS handles everything
behind the scenes.

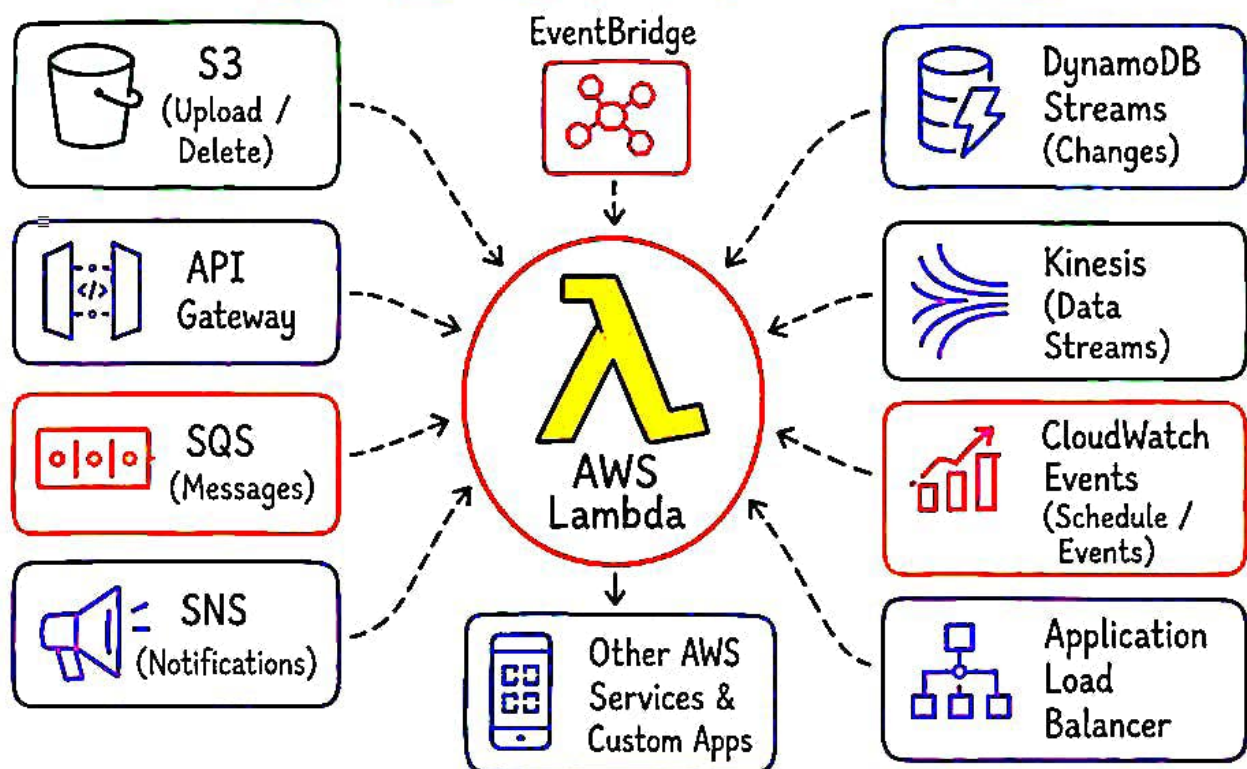
COMMON TRIGGERS

- API Gateway
- S3
- SQS
- SNS
- EventBridge
- DynamoDB Streams
- Kinesis
- CloudWatch Events
- Application Load Balancer
- And more...

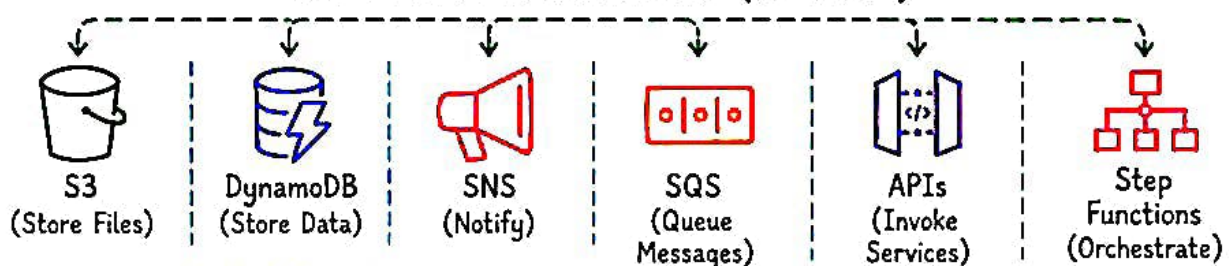


④ LAMBDA TRIGGERS & INTEGRATIONS

Lambda can be triggered by many AWS services and can integrate with various services to perform actions.



COMMON INTEGRATIONS (ACTIONS)



KEY TAKEAWAY

Lambda connects with many services, reacts to events, and drives **actions**!

EVENTS
(Many Sources)



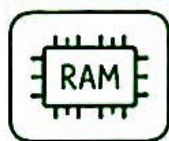
ACTIONS
(Many Destinations)

One Function. Infinite Possibilities.

⑤ LAMBDA CONFIGURATION

Lambda gives you **control** over how your function runs, what it can access, and how it **behaves**.

WHAT CAN YOU CONFIGURE?



MEMORY

128 MB to 10,240 MB.
More memory = More CPU.



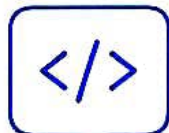
TIMEOUT

Set the maximum time your function can run (max 15 min).



EXECUTION ROLE

IAM role that defines what AWS resources Lambda can access.



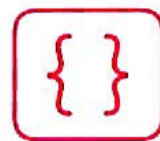
RUNTIME

Choose the language runtime (Python, Node.js, Java, Go, .NET, Ruby, etc.)



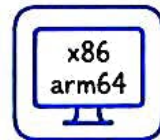
LAYERS

Share common code, libraries, or dependencies across multiple functions.



ENVIRONMENT VARIABLES

Store key-value pairs that your code can use at runtime.



ARCHITECTURE

Choose between x86_64 (Intel/AMD) or arm64 (Graviton).



VPC CONFIGURATION

Connect Lambda to your VPC (Subnets, Security Groups) to access private resources.



EPHEMERAL STORAGE

Default 512 MB (/tmp).
Can be increased up to 10 GB.



KMS ENCRYPTION

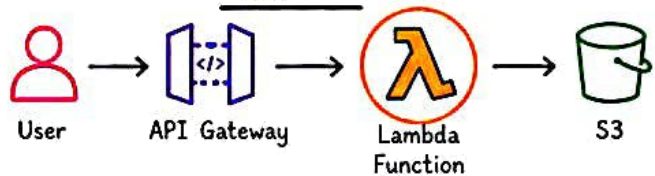
Encrypt environment variables or data at rest using AWS KMS.



IMPORTANT

- Execution Role = **Permissions**
- Timeout = Prevents long running functions
- Memory = Affects performance & cost
- Use least privilege for IAM Role
- Keep environment variables secure

EXAMPLE



You manage the configuration.
AWS manages the **infrastructure**.

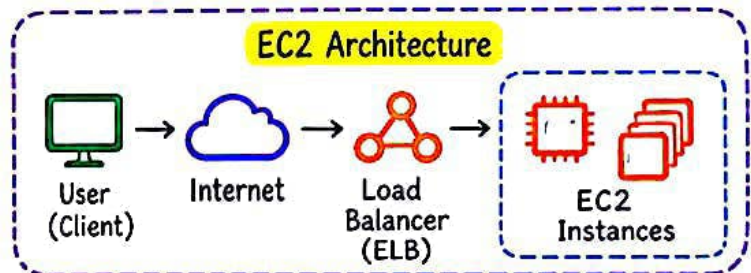
CONFIGURE SMARTLY. RUN EFFICIENTLY. PAY ONLY FOR **WHAT YOU USE**.

aws ⑥ EC2 SIMPLIFIED

EC2 (Elastic Compute Cloud) lets you create and manage virtual servers (instances) in the AWS cloud.

KEY CONCEPTS

- ✓ Instances - Virtual servers
- ✓ AMI - Amazon Machine Image
- ✓ Key Pair - Secure SSH access
- ✓ Security Group - Virtual firewall
- ✓ EBS - Elastic Block Store (storage)



INSTANCE TYPES

Family	Use Case
t2 / t3	General Purpose
m5 / m6	Compute Optimized
c5	Memory Optimized
r5	Storage Optimized

COMMON EC2 OPERATIONS

- ✓ Launch Instance - `aws ec2 run-instances`
- ✓ Connect (SSH) - `ssh -i key.pem ec2-user@...`
- ✓ Stop Instance - `aws ec2 stop-instances`
- ✓ Start Instance - `aws ec2 start-instances`
- ✓ Terminate Instance - `aws ec2 terminate-instances`

BEST PRACTICES

- ✓ Use least privilege (IAM)
- ✓ Keep AMIs updated
- ✓ Use Security Groups (not open 0.0.0.0/0)
- ✓ Right-size instances
- ✓ Enable CloudWatch monitoring

USE CASES

-  Web applications
-  Databases
-  Application servers
-  DevOps tooling

 **Key Takeaway:** EC2 gives you **scalable, flexible** and **on-demand** compute power in the AWS cloud.

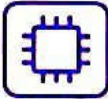
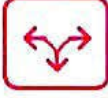
Learn • Practice • Build • Deploy

Master AWS.
Build Real-World
DevOps Skills!

aws 7 EC2 PRICING

You pay for EC2 based on the **instance type**, **usage**, and **pricing model** you choose.

FACTORS THAT AFFECT PRICE

-  **Instance Type**
CPU, Memory, Storage, Network.
-  **Usage Duration**
Hourly billing - pay for what you use.
-  **Region**
Prices vary by AWS Region.
-  **Storage**
EBS volume type and size.
-  **Data Transfer**
Outbound data costs extra.

EC2 PRICING OPTIONS

Option	Best For
 On-Demand Instances	Short-term, unpredictable workloads
 Reserved Instances	Long-term workloads (1 or 3 years)
 Savings Plans	Flexible savings on compute usage
 Spot Instances	Fault-tolerant, interruptible workloads (up to 90% off)
 Dedicated Hosts	Full control, compliance requirements

COST OPTIMIZATION TIPS

- ✓ Use Right-Sized Instances
- ✓ **Stop** Unused Instances
- ✓ Use Reserved Instances or Savings Plans
- ✓ Use **Spot Instances** for non-critical workloads
- ✓ Delete Unused EBS Volumes and Snapshots
- ✓ Monitor with AWS **Cost Explorer** & **Budgets**
- ✓ Use Auto Scaling to match demand

MONITOR YOUR COSTS

-  Use AWS Cost Explorer
-  Set Budgets & Alerts
-  Review Unused Resources
-  Analyze & Optimize Regularly

KEY TAKEAWAY:

EC2 is flexible and powerful. Choose the right instance, right pricing model, and continuously optimize to **save more!**

☆ ☆ Right Choice. Right Size. Right Price. ☆ ☆

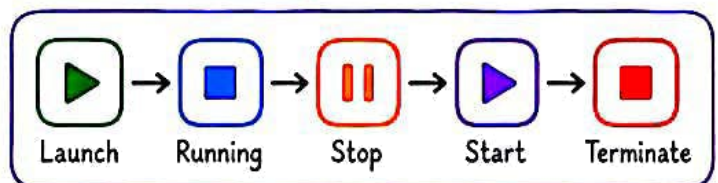
aws ⑧ EC2 RECAP & SUMMARY

Amazon EC2 gives you **resizable** compute capacity in the cloud.
You launch **virtual servers (instances)** and pay only for what you use.

WHAT WE LEARNED

- ✓ What is EC2 & how it works
- ✓ EC2 architecture & key components
- ✓ Instance types & use cases
- ✓ Launching & connecting to an instance
- ✓ Storage options (EBS, AMI, Snapshots)
- ✓ Security Groups & key pair access
- ✓ Monitoring & troubleshooting basics
- ✓ Pricing models & cost optimization

EC2 LIFE CYCLE



COMMON USE CASES



EC2 BEST PRACTICES

- ★ Use IAM roles - avoid access keys
- ★ Keep AMIs updated
- ★ Use least privilege access
- ★ Use Security Groups (not open 0.0.0.0/0)
- ★ Right-size instances
- ★ Enable CloudWatch monitoring
- ★ Delete unused resources
- ★ Use Auto Scaling for high availability
- ★ Regular backups (Snapshots)

QUICK COMMANDS (AWS CLI)

```
aws ec2 run-instances --image-id ami-12345678 \
  --instance-type t2.micro --key-name my-key
aws ec2 start-instances --instance-ids i-12345678
aws ec2 stop-instances --instance-ids i-12345678
aws ec2 describe-instances
aws ec2 terminate-instances --instance-ids i-12345678
```



KEY TAKEAWAY: EC2 is the backbone of AWS compute.
Understand it well, and you can **build**, **scale**
and **run** anything in the cloud!



Practice Today, Build Tomorrow, Deploy Anywhere!

Keep Learning
Keep Building
Keep Growing! ❤️

aws ⑨ EC2 HIGH AVAILABILITY

High Availability ensures your applications stay running, even if something fails.

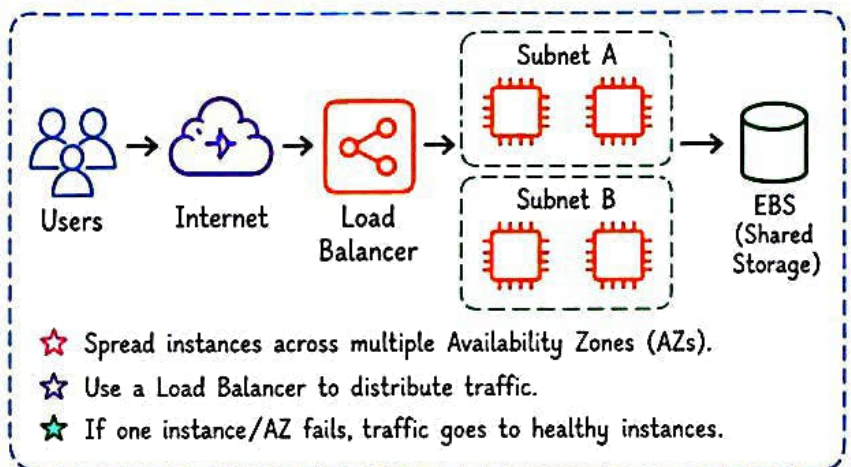
WHY HIGH AVAILABILITY?

- ✓ Minimize downtime
- ✓ Handle failures automatically
- ✓ Provide a better user experience
- ✓ Scale to meet demand

KEY COMPONENTS

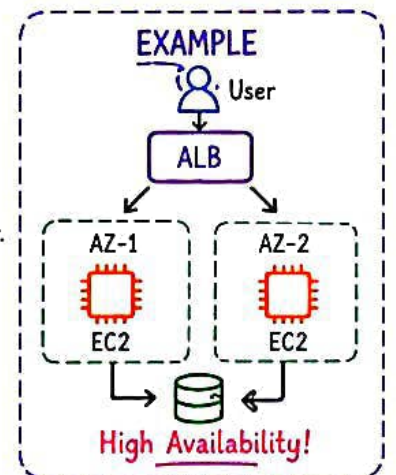
-  **EC2 Instances**
Virtual servers running your applications.
-  **Load Balancer**
Distributes traffic across healthy instances.
-  **Auto Scaling Group**
Automatically adds or removes instances based on demand.
-  **Security Groups**
Act as virtual firewalls to control traffic.
-  **EBS Volumes**
Provides persistent storage for your instances.

EC2 HIGH AVAILABILITY ARCHITECTURE



BEST PRACTICES

- ✓ Use at least 2 Availability Zones.
- ✓ Enable Auto Scaling for elasticity.
- ✓ Use Health Checks with Load Balancer.
- ✓ Keep AMIs and instances updated.
- ✓ Monitor with CloudWatch.
- ✓ Backup important data regularly.



KEY TAKEAWAY: High Availability is not just about having backups, it's about designing for failure and automatic recovery.



★ Design for Failure. Build for Reliability. Deliver with Confidence. ★

Keep Learning.
Keep Building.
Keep Growing! ❤️

aws 10 EC2 SUMMARY & CHEAT SHEET

EC2 gives you the power to run your workloads in the cloud.
It's **flexible**, **scalable** and **secure** when used the right way.

TOP EC2 USE CASES

-  **Web Applications**
Host websites and web apps.
-  **Databases**
Run relational or NoSQL databases.
-  **Application Servers**
Run backend services and APIs.
-  **Dev / Test Environments**
Create isolated environments.
-  **Batch Processing**
Run scheduled or heavy workloads.

QUICK TIPS

- ★ Use IAM roles instead of keys.
- ★ Keep AMIs updated.
- ★ Use Security Groups, not NACLs for instance-level rules.
- ★ Enable CloudWatch monitoring.
- ★ Use Auto Scaling for scalability.
- ★ Regularly backup with Snapshots.

EC2 RECAP CHECKLIST

-   Launch Instance
-   Configure Security
-   Attach Storage
-   Connect & Test
-   Monitor Performance
-   Scale as Needed
-   Optimize Costs
-   Backup & Recover

KEY TAKEAWAY:

EC2 is powerful, but the real **magic** comes from how you **configure**, **secure**, and **optimize** it!

EC2

Learn • Practice • Build • Deploy

AWS INTERVIEW QUESTIONS

- ① What is EC2 and how does it work?
- ② What are the different EC2 instance types?
- ③ How do you secure an EC2 instance?
- ④ What is the difference between **On-Demand**, **Reserved**, and **Spot** Instances?
- ⑤ How do you achieve High Availability in EC2?



COMMON ISSUES & HOW TO FIX

Issue	Possible Cause	How to Fix
 Instance not reachable	Security Group / NACL Incorrect	Check SG rules & NACL settings.
 High CPU Utilization	Traffic spike / Poorly optimized app	Scale up / out or optimize the app.
 High Cost	Idle instances / Wrong instance type	Right-size instances & stop unused ones.
 Slow Performance	EBS type / Low IOPS	Use gp3/io2 EBS or increase IOPS.

Keep Learning,
Keep Building,
Keep Growing! ❤️